Informe de Normalización y Estructura de Datos

Sistema de Matriculación - Universidad Central del Ecuador (UCE)

---

1. Introducción y Contexto

El presente documento analiza la estructura del sistema de matriculación desarrollado en FastAPI y SQLite para la Universidad Central del Ecuador (UCE). Se presenta el proceso formal de normalización de la base de datos hasta la Tercera Forma Normal (3FN), garantizando la integridad referencial, la eliminación de redundancias y la optimización de consultas.

2. Proceso de Normalización

La normalización es una técnica de diseño de bases de datos relacionales que reduce la redundancia y evita anomalías de inserción, actualización y borrado.

Primera Forma Normal (1FN) - Atomicidad y Claves Primarias

Para cumplir con la 1FN:
• Todos los atributos contienen valores atómicos e indivisibles (ej. nombre, apellido, cédula, fecha de nacimiento).
• Cada celda almacena un solo valor por registro.
• Cada tabla posee una Clave Primaria (Primary Key) única e incremental (**id_paralelo**, **id_materia**, **id_usuario**).

Segunda Forma Normal (2FN) - Dependencia Funcional Completa

Para cumplir con la 2FN:
• La base de datos ya se encuentra en 1FN.
• Todas las tablas poseen claves primarias simples de una sola columna (llaves sustitutas autoincrementables).
• Todos los atributos no clave dependen de forma completa de la totalidad de la clave primaria, eliminando dependencias parciales.

Tercera Forma Normal (3FN) - Eliminación de Dependencias Transitivas

Para cumplir con la 3FN:
• La base de datos cumple con la 2FN.
• Ningún atributo no clave depende transitivamente de otro atributo no clave.
• La información del paralelo (como *nombre_paralelo*) se extrajo a su propia entidad (**paralelos**). La tabla **usuarios** solo conserva la Clave Foránea (**id_paralelo**), garantizando que cambiar el nombre de un paralelo no requiera modificar múltiples registros de alumnos.

3. Modelo Entidad-Relación y Tablas de la Base de Datos

Tabla 1: paralelos

Campo	Tipo de Dato	Restricción	Descripción
id_paralelo	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identificador único del paralelo
nombre_paralelo	TEXT	NOT NULL, UNIQUE	Código o nombre del paralelo (001, 002, 003)

Tabla 2: materias

Campo	Tipo de Dato	Restricción	Descripción
id_materia	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identificador único de la materia
nombre_materia	TEXT	NOT NULL, UNIQUE	Nombre oficial de la asignatura (ej. base de datos)

Tabla 3: usuarios

Campo	Tipo de Dato	Restricción	Descripción
id_usuario	INTEGER	PRIMARY KEY, AUTOINCREMENT	ID único del estudiante
nombre	TEXT	NOT NULL	Nombres del estudiante
apellido	TEXT	NOT NULL	Apellidos del estudiante
edad	INTEGER	NOT NULL	Edad en años cumplidos
cedula	TEXT	NOT NULL, UNIQUE	Número de cédula de identidad
fecha_nacimiento	TEXT	NOT NULL	Fecha de nacimiento (AAAA-MM-DD)
id_paralelo	INTEGER	FOREIGN KEY (paralelos)	Clave foránea hacia la

			tabla de paralelos

4. Arquitectura de la API (FastAPI) y Operaciones CRUD

La aplicación expone una API RESTful estructurada mediante modelos de validación Pydantic y SQLite como motor de base de datos relacional:

• **GET /paralelos:** Obtiene la lista completa de paralelos disponibles (001, 002, 003).
• **GET /materias:** Consulta las 6 materias registradas en el sistema.
• **GET /usuarios:** Lista todos los estudiantes realizando un LEFT JOIN con la tabla paralelos para incluir la denominación del paralelo.
• **POST /usuarios:** Registra un nuevo estudiante validando la unicidad de cédula e integridad de FK.
• **PUT /usuarios/{id_usuario}:** Actualiza de forma completa la información de un estudiante existente.
• **DELETE /usuarios/{id_usuario}:** Elimina un estudiante especificando su identificador único.

> **Conclusión de Diseño:**
> El esquema implementado cumple de forma estricta con la 3FN, asegurando la separación clara de responsabilidades entre paralelos, materias y usuarios. Esto previene redundancias de datos y garantiza una alta escalabilidad para la API en FastAPI.